



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingenieria de Sistemas

Proyecto Simulador de Bases de Datos

Curso: **Calidad y Pruebas de Software**

Docente: **MAG. Patrick Cuadros Quiroga**

Integrantes:

- **Jhony Vargas Luque (2022075754)**
- **Abel Fernando Pacompia Ortiz (2023076797)**

Tacna - Peru

2026

Informe de Factibilidad

Version: 2.1

Version	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-04-05	Version inicial
2.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-06-21	Actualizacion segun implementacion final del simulador
2.1	APO, JVL	APO, JVL	P. Cuadros Q.	2026-07-04	Actualizacion segun codigo actual, GitHub Actions y despliegue

INDICE GENERAL

1. [Descripcion del proyecto](#)
2. [Alcance actual](#)
3. [Riesgos](#)
4. [Factibilidad tecnica](#)
5. [Factibilidad economica](#)
6. [Factibilidad operativa](#)
7. [Factibilidad legal](#)
8. [Factibilidad social y ambiental](#)
9. [Conclusiones](#)

1. Descripcion del proyecto

Simulador de Bases de Datos es una aplicacion web y desktop orientada al aprendizaje y practica de consultas SQL y NoSQL sin instalar motores reales de base de datos. El sistema ofrece un entorno tipo IDE con editor Monaco, multiples pestanas de consulta, explorador de esquema, panel de resultados, importacion/exportacion de datos y simulacion de carga.

El proyecto permite trabajar con sintaxis y comportamientos representativos de **SQL Server, MySQL, PostgreSQL, Oracle, SQLite, MongoDB y Redis**. La ejecucion se realiza principalmente en memoria mediante **AlaSQL**, con persistencia local en **IndexedDB** para tablas importadas o creadas, y servicios de apoyo con **Firebase** para autenticacion, presencia de usuarios, monitoreo de sesiones del simulador y administracion de roles.

2. Alcance actual

La version actual incluye:

- Editor de consultas con Monaco Editor.

- Ejecucion de consultas SQL por motor relacional simulado.
- Soporte de operaciones simuladas para MongoDB y Redis.
- Preprocesamiento de scripts SQL Server para compatibilidad con el motor en memoria.
- Importacion de archivos `.sql` , `.csv` y `.json` .
- Exportacion de resultados en CSV, JSON y Excel.
- Exportacion de esquemas y bases completas segun el motor seleccionado.
- Persistencia de tablas y metadatos en IndexedDB.
- Explorador de esquema con vista de tablas y previsualizacion de datos.
- Historial de consultas y bitacora de ejecucion.
- Simulacion de latencia, errores aleatorios, limites de conexion y niveles de aislamiento.
- Simulador de carga con TPS, latencia, CPU estimada, errores, comparacion entre motores y modo progresivo.
- Panel administrativo para monitoreo de sesiones y gestion de usuarios.
- Version web con Vite y version desktop con Electron.
- Landing publica en `landing/` y despliegue automatico a GitHub Pages.
- Workflows de GitHub Actions para validar rendimiento y publicar la landing.

Fuera de alcance:

- Conexion directa a motores reales de base de datos.
- Garantia de compatibilidad total con todos los dialectos SQL.
- Ejecucion de procedimientos almacenados, funciones de usuario o CTEs complejas.
- Benchmark real de rendimiento de motores productivos.
- Persistencia empresarial multiusuario de datos operacionales.
- Uso como herramienta de administracion de bases de datos reales.

Por ello, el sistema debe considerarse un **simulador academico**, no un cliente profesional de base de datos ni una plataforma de benchmarking real.

3. Riesgos

Riesgo	Descripcion	Mitigacion
Diferencias de dialecto SQL	Algunas sentencias pueden no ejecutarse igual que en motores reales.	Preprocesadores por motor, mensajes de error con sugerencias y documentacion de limitaciones.
Expectativa de conexion real	El usuario podria asumir que la aplicacion se conecta a servidores reales.	Indicar claramente que la ejecucion es simulada/en memoria.
Perdida de datos locales	Al limpiar IndexedDB o cambiar de navegador se pueden perder tablas persistidas.	Exportacion de sesiones, esquemas y bases completas.
Dependencia de Firebase	Funciones de presencia, autenticacion y admin dependen de configuracion externa.	Degradacion controlada cuando Firebase no esta configurado.

Riesgo	Descripcion	Mitigacion
Complejidad de UI	Multiples herramientas pueden saturar al usuario principiante.	Interfaz por modales, ayuda integrada, plantillas y valores por defecto.
Resultados de carga simulados	TPS, CPU y latencia son estimaciones, no mediciones reales de motores externos.	Presentar el modulo como simulador didactico de carga.

4. Factibilidad tecnica

El proyecto es tecnicamente viable porque utiliza tecnologias consolidadas, gratuitas y compatibles con ejecucion local:

Tecnologia	Uso
React 18 + TypeScript	Componentes de interfaz y logica de aplicacion.
Vite	Servidor de desarrollo y empaquetado web.
Tailwind CSS	Estilos utilitarios y sistema visual.
Monaco Editor	Editor de consultas con experiencia similar a VS Code.
AlaSQL	Motor SQL en memoria para ejecucion local.
IndexedDB	Persistencia local de tablas y esquemas.
Zustand	Gestion de estado global.
Firebase Auth/Realtime Database	Autenticacion, presencia, roles y monitoreo en tiempo real.
Electron	Empaquetado como aplicacion desktop.
SheetJS	Exportacion de resultados a Excel.
GitHub Actions	Automatizacion de build, pruebas de rendimiento y despliegue de landing.

El repositorio ya cuenta con scripts de ejecucion y construccion:

- `npm run dev` : inicia la version web.
- `npm run build` : compila TypeScript y genera build Vite.
- `npm run preview` : previsualiza el build.
- `npm run test:performance` : ejecuta la prueba automatica de carga del simulador.
- `npm run electron:dev` : ejecuta modo desktop en desarrollo.
- `npm run electron:build` : genera instaladores desktop.

Ademas, el repositorio contiene los workflows:

Workflow	Archivo	Funcion
Database Load Performance	<code>.github/workflows/performance.yml</code>	Compila el proyecto, ejecuta pruebas de rendimiento por motor y escenario, genera artifacts y resumen consolidado.

Workflow	Archivo	Funcion
Deploy Landing Page	<code>.github/workflows/pages.yml</code>	Publica el contenido de <code>landing/</code> en GitHub Pages.

La prueba de rendimiento automatizada usa 7 motores y 3 escenarios: `light`, `medium` y `heavy`. Cada ejecucion valida latencia promedio, latencia p95, TPS y tasa de errores mediante umbrales definidos por motor.

Resultado: Factibilidad tecnica alta.

5. Factibilidad economica

El proyecto no requiere licencias comerciales. Las herramientas principales son open-source o cuentan con niveles gratuitos suficientes para un entorno academico.

Concepto	Costo estimado
Herramientas de software	S/. 0.00
Equipo de desarrollo propio	S/. 0.00
Internet y energia	S/. 130.00
Tiempo de desarrollo academico	S/. 1600.00
Materiales y documentacion	S/. 275.00
Total estimado	S/. 2005.00

Frente a instalar y administrar siete motores reales de base de datos, el simulador reduce costos de instalacion, configuracion y mantenimiento para fines de aprendizaje.

Resultado: Factibilidad economica alta.

6. Factibilidad operativa

El sistema es operable por estudiantes con conocimientos basicos de bases de datos. La aplicacion presenta un flujo guiado: inicio de sesion, pantalla de bienvenida, editor, pestanas por motor, explorador de esquema y panel de resultados.

El uso recomendado en modo web es:

1. Ejecutar `npm install`.
2. Ejecutar `npm run dev`.
3. Abrir la URL local generada por Vite.
4. Iniciar sesion o registrarse.
5. Importar datos o crear tablas desde el editor.
6. Ejecutar consultas, revisar resultados y exportar evidencias.

El modulo de simulacion de carga permite ejecutar pruebas controladas sin requerir infraestructura real de base de datos.

Resultado: Factibilidad operativa alta.

7. Factibilidad legal

El proyecto utiliza dependencias de uso comun en aplicaciones web y desktop academicas. Los datos se procesan localmente en el navegador, salvo funciones de autenticacion, presencia y administracion cuando Firebase esta configurado.

El sistema no almacena informacion sensible por defecto. Los datos tratados pueden incluir:

- Consultas escritas por el usuario.
- Tablas importadas en el navegador.
- Resultados de consultas.
- Metadatos de sesiones y usuarios para funciones colaborativas.
- Historial local de consultas y pruebas de carga.

Se recomienda no importar datos reales sensibles sin autorizacion y mantener las credenciales de Firebase fuera del repositorio publico.

Resultado: Factibilidad legal alta si se respeta el uso academico y la proteccion de datos.

8. Factibilidad social y ambiental

El proyecto tiene impacto social positivo porque facilita el aprendizaje practico de bases de datos, consultas SQL/NoSQL, diferencias entre motores y conceptos de rendimiento sin requerir instalaciones complejas. Al ser software, no genera residuos fisicos ni demanda infraestructura adicional significativa.

Resultado: Factibilidad social y ambiental alta.

9. Conclusiones

El **Simulador de Bases de Datos** es viable desde el punto de vista tecnico, economico, operativo, legal, social y ambiental. La version actual cumple con el objetivo academico de ofrecer un entorno accesible para practicar consultas, importar datos, visualizar esquemas, exportar resultados y simular carga en diferentes motores.

Se recomienda presentar el sistema como herramienta didactica, aclarando que no reemplaza a motores reales ni a clientes profesionales de administracion de bases de datos.